Re-implementing "Were RNNs All We Needed?" (minGRU and minLSTM) at Reduced Scale

Ashutosh Sharma
B.Tech CSE, RGIPT
24cs3013@rgipt.ac.in

***Abstract*—Recurrent networks cannot be trained in parallel over time because the gates of an LSTM or GRU read the previous hidden state. Feng et al. [1] remove that dependency, obtaining minGRU and minLSTM, whose recurrence is linear and can be evaluated with a parallel prefix scan. We re-implement both models from scratch in PyTorch with a sequential loop, a doubling scan and a numerically stable log-space scan, and reproduce selected results of [1] at reduced scale on one Tesla T4 GPU. The three modes agree to within 6×10^{-15} in output and 2×10^{-12} in gradients (float64). On a speed benchmark the scan trains $3.1\times$ (minGRU) and $2.6\times$ (minLSTM) faster than a loop-based baseline at length 512 and $9.1\times$ (minGRU) at length 2048, far below the $175\times$ and $235\times$ reported in the paper, and uses about $4\times$ more memory. On Selective Copy with 5,000 training steps, minGRU reaches 26, 40 and 46% with 1, 2 and 3 layers, against 37, 97 and 99.5% in the paper after 400,000 steps. On TinyShakespeare, minGRU, GRU and a Transformer reach similar best validation losses (1.495, 1.462, 1.494). We report where our numbers match the paper, where they do not, and the likely causes.**

***Index Terms*—recurrent neural networks, parallel scan, minGRU, minLSTM, reproducibility**

I. Introduction

LSTMs [2] and GRUs [3] compute the hidden state h_t from h_{t-1} inside their gates, so training must run T steps one after another through time (backpropagation through time). Transformers [4] avoid this by processing all positions at once, at the cost of quadratic attention. Feng et al. [1] ask whether the old recurrent models can be made parallel and keep their constant per-step cost at inference. Their answer is to drop h_{t-1} from the gates and the $\tanh$ from the candidate, which turns the update into a linear recurrence that a parallel scan [5], [6] solves in $O(\log T)$ rounds.

This report documents a from-scratch re-implementation of that idea and a reduced-scale reproduction of three results of [1]: (i) the minimal models have far fewer parameters, (ii) they train much faster than loop-based GRU and LSTM, and (iii) they match the accuracy of standard models on Selective Copy and character-level language modelling. Our contributions are:

- An implementation of minGRU and minLSTM with three evaluation modes (loop, doubling scan, log-space scan) and tests that the modes give the same outputs and gradients.
- A worked derivation of the scan, including the associativity of the pair rule, with a hand-computed four-step example.
- A reduced-scale comparison against GRU, LSTM and a small Transformer on Selective Copy, TinyShakespeare and a speed and memory benchmark, with the paper's numbers alongside ours and an account of the differences.

The code, notebook and a browser demo of the equivalence check are available at the repository linked in Section III.

II. Method

A. Original and minimal gates

The GRU uses an update gate z_t, a reset gate r_t and a $\tanh$ candidate, all reading h_{t-1}:

$$z_t = \sigma(\mathrm{Lin}_z([x_t, h_{t-1}])), \quad r_t = \sigma(\mathrm{Lin}_r([x_t, h_{t-1}])), \tag{1}$$

$$\tilde{h}_t = \tanh(\mathrm{Lin}_h([x_t,\, r_t \odot h_{t-1}])), \tag{2}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \tag{3}$$

minGRU removes the reset gate and every dependence on h_{t-1} inside z_t and $\tilde{h}_t$, and drops the $\tanh$:

$$z_t = \sigma(\mathrm{Lin}_z(x_t)), \quad \tilde{h}_t = \mathrm{Lin}_h(x_t), \tag{4}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \tag{5}$$

The LSTM has forget, input and output gates, a $\tanh$ candidate and a separate cell state c_t. minLSTM keeps a single state, removes the output gate, and normalises the two remaining gates so that they sum to one:

$$f_t = \sigma(\mathrm{Lin}_f(x_t)), \quad i_t = \sigma(\mathrm{Lin}_i(x_t)), \quad \tilde{h}_t = \mathrm{Lin}_h(x_t), \tag{6}$$

$$f'_t = \frac{f_t}{f_t + i_t}, \quad i'_t = \frac{i_t}{f_t + i_t}, \tag{7}$$

$$h_t = f'_t \odot h_{t-1} + i'_t \odot \tilde{h}_t. \tag{8}$$

Both updates have the form

$$h_t = a_t \odot h_{t-1} + b_t, \tag{9}$$

with $a_t = 1 - z_t,\ b_t = z_t \odot \tilde{h}_t$ for minGRU and $a_t = f'_t,\ b_t = i'_t \odot \tilde{h}_t$ for minLSTM. Because a_t and b_t depend only on x_t, all T pairs (a_t, b_t) can be computed at once. The hidden state is still a function of the whole past, since it is built from Eq. (9).

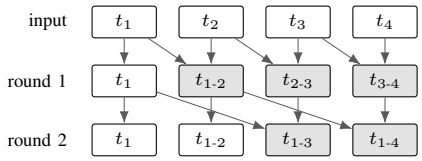


Fig. 1. Doubling scan on four steps. Each shaded cell merges two cells of the row above with the rule in Eq. (11): round 1 joins neighbours, round 2 joins cells two places apart, so $\lceil \log_2 4 \rceil = 2$ rounds suffice. With the hand example $x = (1, -0.5, 2, 0.5)$ the scan gives $h = (0.621,\ 0.191,\ 1.530,\ 0.883)$, identical to the loop.

B. Parallel scan and associativity

Write a step as the pair $p_t = (a_t, b_t)$, which stands for the function $h \mapsto a_t h + b_t$. Applying p_1 then p_2 gives

$$a_2(a_1 h_0 + b_1) + b_2 = (a_1 a_2)\, h_0 + (a_2 b_1 + b_2), \tag{10}$$

which is again a function of the same form. This defines the pair rule

$$(a_1, b_1) \circ (a_2, b_2) = (a_1 a_2,\ \ a_2 b_1 + b_2). \tag{11}$$

The rule is associative:

$$\begin{aligned}(p_1 \circ p_2) \circ p_3 &= (a_1 a_2 a_3,\ \ a_3(a_2 b_1 + b_2) + b_3) \\ &= (a_1 a_2 a_3,\ \ a_2 a_3 b_1 + a_3 b_2 + b_3) \\ &= p_1 \circ (p_2 \circ p_3),\end{aligned} \tag{12}$$

as it is the composition of functions. It is not commutative. If $(A_t, B_t) = p_1 \circ \cdots \circ p_t$, then $h_t = A_t h_0 + B_t$, so all hidden states are the prefix products of $p_1, \ldots, p_T$ under $\circ$.

Associativity lets us choose the order in which the merges are done. The sequential loop uses $((p_1 \circ p_2) \circ p_3) \circ p_4$, where each merge waits for the previous one ($T - 1$ merges in a row). The doubling scan [6] merges, in round k, every element with the one 2^{k-1} places to its left, all at once. After $\lceil \log_2 T \rceil$ rounds every position holds its full prefix (Fig. 1). For example, $p_1 \circ p_2 \circ p_3$ is computed as $p_1 \circ (p_2 \circ p_3)$, which equals the loop's grouping only because of Eq. (11)'s associativity. The doubling scan does $O(T \log T)$ work in $O(\log T)$ depth, against $O(T)$ work and depth for the loop.

C. Log-space scan

The paper trains with a log-space scan [1], [7] for numerical stability, which requires $b_t > 0$ and $h_0 > 0$. The candidate is therefore passed through

$$g(x) = \begin{cases} x + 0.5 & x \geq 0 \\ \sigma(x) & x < 0 \end{cases} \tag{13}$$

and all quantities are computed from logits with $\log \sigma(k) = -\operatorname{softplus}(-k)$ and $\log(1 - \sigma(k)) = -\operatorname{softplus}(k)$. For minGRU, $\log a_t = -\operatorname{softplus}(k_t)$ and $\log b_t = -\operatorname{softplus}(-k_t) + \log g(\tilde{h}_t)$. For minLSTM, with $\delta_t = \operatorname{softplus}(-p_t^f) - \operatorname{softplus}(-p_t^i)$ we have $\log f'_t = -\operatorname{softplus}(\delta_t)$ and $\log i'_t = -\operatorname{softplus}(-\delta_t)$. Writing $a^*_t = \sum_{s \leq t} \log a_s$,

$$\log h_t = a^*_t + \operatorname{logcumsumexp}_{s \leq t}(\log b_s - a^*_s)\,, \tag{14}$$

where $\log h_0$ is prepended as an extra element when $h_0 \neq 0$. Because of g, the log-space model is a different function from the model with a linear candidate; comparisons of loop and scan are therefore only made within the same variant.

III. Implementation and Setup

Code. Everything is in one PyTorch notebook: `scan_loop`, `scan_tree` (Eq. (11)) and `scan_log` (Eq. (14)); `MinGRU` and `MinLSTM` modules with a `parallel` switch and a log-space or linear-candidate option; a residual block (LayerNorm, minimal RNN, down-projection, and for language modelling a depth-wise Conv4 and an MLP); and the baselines. The code was written from the paper; the log-space scan follows the formulation of [7] described in the paper's appendix. Repository: https://github.com/⟨username⟩/Were-RNNs-All-WeNeeded-miniLSTM-miniGRU- (notebook `minrnn_notebook.ipynb`, with README, seeds and a demo app).

Baselines. GRU and LSTM are used in two forms: a Python loop over `GRUCell`/`LSTMCell` (the sequential baseline for speed) and the fused cuDNN `nn.GRU`/`nn.LSTM` (for accuracy and as a stronger speed reference). The Transformer is a pre-norm causal decoder with learned positions and `scaled_dot_product_attention`. All models share the same wrapper (embedding, blocks, LayerNorm, linear head).

Tests. In float64 (batch 3, length 33, 5 inputs, 8 hidden units, with and without h_0, both variants) the notebook asserts that loop and scan give the same outputs and the same parameter gradients to 10^{-8}, that changing future inputs does not change past outputs (causality), and that the state is bounded by the largest candidate magnitude when $h_0 = 0$. The scan primitives are tested separately for $T \in \{1, 2, 7, 64, 257\}$.

Experimental setup. Table I compares our setup with the paper's. All runs used one Tesla T4 with PyTorch 2.11 and CUDA 13.0. Selective Copy used seeds 0, 1 and 2 with Adam (AdamW with zero weight decay); the language model (AdamW, weight decay 0.01) and the benchmark used a single run.

Selective Copy. Sequences of length L contain 16 data tokens (14 possible symbols) at random positions among noise tokens; the last 16 positions hold a marker and the model must output the data tokens in order. Accuracy is token accuracy on those 16 positions on a fixed set of 512 sequences, so chance is $1/14 \approx 7.1\%$. We report the best of ten evaluations per run.

Language modelling. Character-level TinyShakespeare [8] (1.1M characters, 65 symbols, 90/10 split). Validation loss is the mean over 20 random batches of 32 sequences, evaluated every 50 steps.

IV. Results

A. Parameters

Counting weight matrices only, the ratio of minimal to standard parameters at hidden expansion $\alpha = 1, 2, 3, 4$ is 33, 22, 17, 13% for minGRU against GRU and 38, 25, 19, 15% for minLSTM against LSTM. These match the paper exactly,

TABLE I
SETUP: ORIGINAL PAPER [1] VERSUS THIS REPRODUCTION.

	Paper	Ours
Hardware	16 GB GPUs (P100, T4, Quadro 5000); speed test on T4	Tesla T4
Selective Copy	$L = 4096$, 16 data tokens, 400k steps, batch 64, lr 3×10^{-4}	$L = 256$, 16 data tokens, 5k steps, batch 64, lr 3×10^{-4}, 3 seeds
Model (copy)	1–3 layers, $d = 64$, expansion 6, dropout 0.1	same
Language model	3 layers, $d = 384$, 5k steps, dropout 0.2	3 layers, $d = 384$, 4k steps, batch 64, block 256, dropout 0.2, AdamW lr 10^{-3}, clip 0.25, 1 seed
Speed test	batch 64, T4	batch 64, $d = 128$, 3 layers, expansion 6, 2 warm-up + 5 timed steps

TABLE II
LARGEST DIFFERENCE, SEQUENTIAL VS. PARALLEL (FLOAT64; WORST CASE OVER RUNS WITH AND WITHOUT h_0).

Module	Variant	$\max\lvert\Delta h\rvert$	$\max\lvert\Delta\nabla\rvert$
scan primitive, $T = 257$	doubling	8.9×10^{-16}	–
scan primitive, $T = 257$	log-space	5.6×10^{-14}	–
minGRU	linear cand.	2.2×10^{-16}	1.4×10^{-14}
minGRU	log-space	5.6×10^{-15}	1.8×10^{-12}
minLSTM	linear cand.	2.2×10^{-16}	1.4×10^{-14}
minLSTM	log-space	4.4×10^{-15}	6.2×10^{-13}

as expected, since the count follows from the equations: $2\alpha d^2$ weights against $3\alpha(1+\alpha)d^2$ for minGRU. Note that the models in our experiments are not parameter-matched: minGRU with expansion 6 has 226k parameters in the 3-layer copy model against 77k for the cuDNN GRU.

B. Loop and scan give the same output

Table II lists the largest differences between the sequential and parallel modes in the tests above. Outputs agree to 10^{-16} for the doubling scan and to 6×10^{-15} for the log-space scan; gradients agree to 10^{-14} and 2×10^{-12}. These are floating-point rounding errors from summing in a different order. This is the central claim of the method: the scan is a different order of the same computation, not an approximation. The hand example of Fig. 1 reproduces $h = (0.621, 0.191, 1.530, 0.883)$ by both routes.

C. Speed and memory

Fig. 2 and Table III show the time of one training step (forward, loss, backward) for batch 64 and 3 layers at $d = 128$ (hidden width 768 for the minimal models). Against the loop-based baseline, minGRU is faster at every length: $3.1\times$ at $L = 512$ and $9.1\times$ at $L = 2048$; minLSTM is $2.6\times$ at 512 and $4.3\times$ at 1024. The speedup grows with length, as the paper predicts, but its size is far below the paper's $175\times$ and $235\times$ at $L = 512$ and $1324\times$ and $1361\times$ at $L = 4096$. At $L = 4096$ minGRU ran out of the T4's memory (and minLSTM from $L = 2048$), so no speedup could be measured there.

TABLE III
TRAINING-STEP TIME AND SPEEDUP OVER THE LOOP-BASED GRU.

	Time per step (ms)			minGRU speedup	
L	GRU (loop)	minGRU	GRU (cuDNN)	ours	paper
128	202	57	8.5	$3.6\times$	–
256	295	112	17.8	$2.6\times$	–
512	688	222	34.5	$3.1\times$	$175\times$
1024	2257	442	66	$5.1\times$	–
2048	8051	890	135	$9.1\times$	–
4096	29671	OOM	279	–	$1324\times$

TABLE IV
SELECTIVE COPY TOKEN ACCURACY (%), MEAN $\pm$ STD OVER 3 SEEDS, BEST OF 10 EVALUATIONS. OURS: $L = 256$, 5K STEPS. PAPER [1]: $L = 4096$, 400K STEPS.

Model	1 layer	2 layers	3 layers
minGRU (ours)	26.0 ± 0.1	39.9 ± 1.3	45.7 ± 1.6
minGRU (paper)	37.0 ± 2.3	96.8 ± 3.2	99.5 ± 0.2
minLSTM (ours)	25.9 ± 0.2	42.9 ± 3.6	36.2 ± 7.0
minLSTM (paper)	37.6 ± 2.0	85.7 ± 5.8	96.0 ± 2.8
GRU, cuDNN (ours)	18.1 ± 0.6	31.4 ± 4.2	42.0 ± 0.9
LSTM, cuDNN (ours)	7.7 ± 0.4	7.5 ± 0.1	7.4 ± 0.0
Transformer (ours)	40.7 ± 1.0	98.1 ± 0.5	99.9 ± 0.0

Three further observations hold in our setting. First, the minimal models were *slower* than the fused cuDNN GRU and LSTM at every length ($6.4\times$ slower than cuDNN GRU at $L = 512$, 222 vs. 34 ms), since our scan is built from eager PyTorch operations rather than a fused kernel. Second, they used about $4.2\times$ the peak memory of the loop GRU at $L = 512$ (3.2 vs. 0.77 GB) and 12.8 GB at $L = 2048$, because the scan materialises intermediate tensors for every step. Third, minGRU's step time grows linearly with L (57, 112, 222, 442, 890 ms for $L = 128, \ldots, 2048$), whereas the loop baseline grows faster than linearly, which inflates the speedup at long lengths (see Section V).

D. Selective Copy

Table IV gives the accuracy after 5,000 steps. Qualitatively the paper's ordering is reproduced: accuracy rises with depth (minGRU $26 \to 40 \to 46\%$; paper $37 \to 97 \to 99.5\%$), and minGRU and minLSTM learn faster than GRU and LSTM under the same budget. The cuDNN LSTM stayed at chance (7.4%) at every depth. Quantitatively we are far from the paper: with 2 and 3 layers our minGRU is 57 and 54 points lower. The Transformer reaches 98.1% and 99.9% with 2 and 3 layers in the same 5,000 steps, which shows that the task and the data pipeline are sound and that the gap is a property of the recurrent models at this budget. In 8 of 9 minGRU runs and in all minLSTM runs the best accuracy was the last evaluation, so the models had not stopped improving; we did not run longer. minLSTM with 3 layers is unstable across seeds (29.8, 35.1, 43.7%).

E. TinyShakespeare

Fig. 3 and Table V show validation loss. All five models reach a similar best loss (1.46 to 1.51 nats per character); the

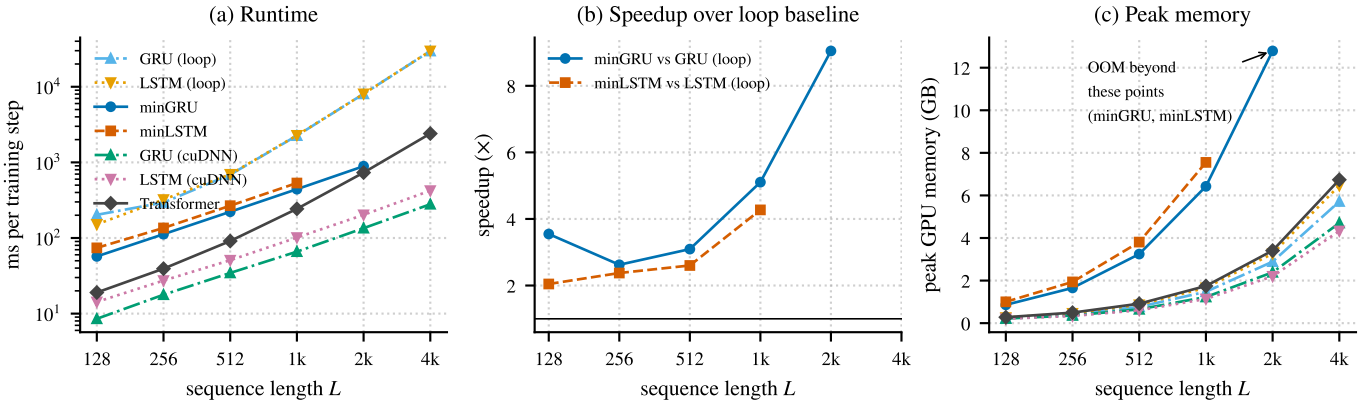


Fig. 2. Speed and memory on a Tesla T4 (batch 64, $d = 128$, 3 layers). (a) Time per training step on a log scale; (b) speedup of the minimal models over the loop-based GRU and LSTM; (c) peak GPU memory; the legend of (a) applies to (a) and (c). minGRU runs out of memory beyond $L = 2048$ and minLSTM beyond $L = 1024$.

TABLE V
TINYSHAKESPEARE, 3 LAYERS, $d = 384$, ONE RUN EACH.

Model	Params	Best val	Step	Val at 4k	Paper
minGRU	6.27M	1.495	700	2.342	1.548
minLSTM	7.15M	1.509	700	2.395	1.555
GRU (cuDNN)	6.26M	1.462	800	1.984	–
LSTM (cuDNN)	7.15M	1.465	650	2.061	–
Transformer	5.47M	1.494	1600	1.627	1.547

cuDNN GRU and LSTM are the best (1.462, 1.465), minGRU and the Transformer are tied (1.495, 1.494), and minLSTM is slightly behind (1.509). The recurrent models reach their best loss early (steps 650 to 800) while the Transformer needs 1600 steps, a ratio of about 2.3× for minGRU (about 3.5× in the paper, steps 575 against 2000). After that, the recurrent models overfit strongly: the validation loss of minGRU rises to 2.34 at step 4000 while its training loss falls to 0.58. With 65 passes over the 1.0M training characters and a 6M-parameter model this is expected; the Transformer degrades much less (1.63).

Our best losses are lower than the paper's (1.548, 1.555, 1.547), but we do not read this as an improvement over the paper. The data split, the number and sampling of validation batches, the stopping point and details of the model differ from the original, and the reported value is the minimum over 80 evaluations on the same validation data, which biases it downwards. The comparison supports only the qualitative claim that the minimal models match the standard ones.

F. Ablations

Table VI collects the ablations. *Depth* is the cleanest: 1, 2 and 3 layers were run with three seeds each and the trend in Table IV is clear for minGRU. We also tried two single-run changes on 2-layer models: a forget-gate bias of $+3$ for minLSTM (46.4% against 42.9% for the default over three seeds) and a linear candidate instead of $g(x)$ for minGRU (49.8% against 39.9%). Both of these were run once, without resetting the seed, and with dropout 0 whereas the reference

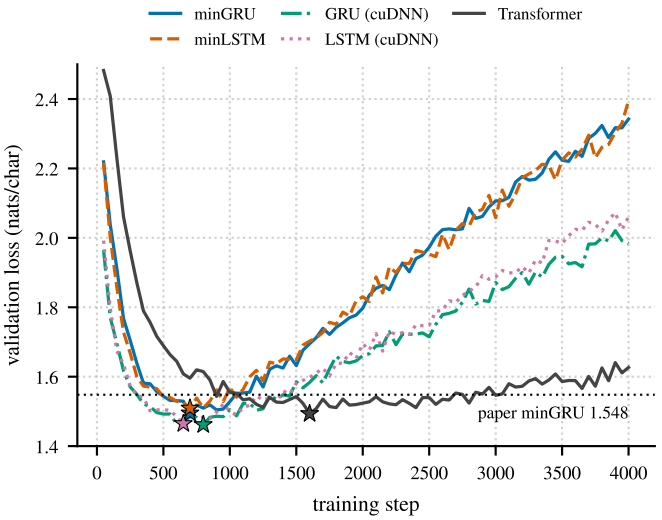


Fig. 3. TinyShakespeare validation loss. Stars mark each model's best value. The dotted line is the paper's minGRU result.

TABLE VI
ABLATIONS ON SELECTIVE COPY (2-LAYER MODELS UNLESS STATED).

Variant	Result (%)	Matched?
Depth 1 / 2 / 3, minGRU	26.0 / 39.9 / 45.7	yes, 3 seeds
minLSTM, forget bias 0 (default)	42.9 ± 3.6	3 seeds
minLSTM, forget bias $+3$	46.4	no, 1 run
minGRU, $g(x)$ candidate (default)	39.9 ± 1.3	3 seeds
minGRU, linear candidate	49.8	no, 1 run

runs used dropout 0.1. They are therefore not matched comparisons and we do not draw conclusions from them. The forget-bias result lies within the spread of the three default seeds (39.8 to 46.8%). The linear-candidate result is suggestive, since it would mean the positivity constraint costs accuracy, but it needs matched seeds and dropout to be confirmed.

V. Discussion and Limitations

What reproduced. The core claims that do not depend on scale are reproduced: the parameter ratios match exactly, and the sequential and parallel evaluations are numerically the same function, including gradients. The qualitative speed result holds as well: the scan is faster than the loop and the advantage grows with length. On TinyShakespeare the minimal models match GRU, LSTM and a Transformer within about 0.05 nats and reach their best loss early. On Selective Copy the paper's ordering by depth holds.

What did not. At $L = 512$ the speedups are about 55 (minGRU) and 90 (minLSTM) times smaller than reported, and Selective Copy accuracy is well below the paper's. For Selective Copy, the likely cause is the training budget, which is 80 times fewer steps on sequences 16 times shorter. The Transformer solving the same task supports this, as does the finding that most runs had not plateaued. For speed we did not profile the code, so the following are hypotheses. (i) Our baseline indexes `h[:, t]` inside the loop, whose backward pass allocates a full-size tensor at every step; this would explain why the loop's time grows faster than linearly in L and would inflate our speedups at long lengths, so the values at $L \leq 512$ (about $3\times$) are the more reliable ones. (ii) The paper does not state its model width, and its 2.7 ms step at $L = 512$ suggests a much smaller model than our 768-wide layers, for which a fixed per-step launch cost dominates and the loop is relatively slower. (iii) Our scan runs as eager PyTorch operations, so it may be limited by memory traffic; this would explain why it is slower than a fused cuDNN kernel and uses several times more memory, and a custom kernel would change it.

Limitations. The language-model and speed results come from one run each; the reported best values are selected on the evaluation data; the Selective Copy sequences are 16 times shorter and the runs 80 times shorter than in the paper; the cuDNN LSTM failing to learn at all suggests a baseline problem we did not diagnose (for example gate initialisation); the Selective Copy models of different families are not parameter-matched; and two of the ablations are not matched comparisons. The paper's own caveat also applies: removing the dependence on h_{t-1} is justified by experiments and not by a proof that expressivity is preserved [9].

VI. Conclusion

We implemented minGRU and minLSTM with sequential, doubling-scan and log-space-scan evaluation and verified that all three give the same outputs and gradients to floating-point precision, which is the property that makes parallel training correct. At reduced scale on one T4 the scan trained 2 to 9 times faster than a loop-based baseline, but with much larger memory use and far below the paper's speedups. Language-modelling quality matched the standard models, while Selective Copy accuracy was well below the paper's because of the small training budget. The main next steps are a longer Selective Copy run, a profile and a fused kernel for the scan, and matched ablations with several seeds.

References

[1] L. Feng, F. Tung, M. O. Ahmed, Y. Bengio, and H. Hajimirsadeghi, "Were RNNs all we needed?," arXiv:2410.01201, 2024.

[2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[3] K. Cho *et al.*, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proc. EMNLP*, 2014.

[4] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017.

[5] G. E. Blelloch, "Prefix sums and their applications," Tech. Rep. CMU-CS-90-190, Carnegie Mellon Univ., 1990.

[6] W. D. Hillis and G. L. Steele Jr., "Data parallel algorithms," *Commun. ACM*, vol. 29, no. 12, pp. 1170–1183, 1986.

[7] F. A. Heinsen, "Efficient parallelization of a ubiquitous sequential computation," arXiv:2311.06281, 2023.

[8] A. Karpathy, "char-rnn," GitHub repository (TinyShakespeare data), 2015. [Online]. Available: https://github.com/karpathy/char-rnn

[9] W. Merrill, J. Petty, and A. Sabharwal, "The illusion of state in state-space models," in *Proc. ICML*, 2024.